

Lab 03 – Sorting Algorithm

Task 01:

Consider we have the sales data of the last quarter of Chase up Company. The dataset represents the total sales (in PKR) generated by 10 different salespeople, and the company needs to sort this data in ascending order for analysis and reporting. Sorting this data will help identify the top and bottom performers and provide insights to improve overall sales strategies.

Sales → {12500, 97000, 8800, 10200, 13040, 7010, 9800, 10150, 12030, 10500}

Your task is to develop a C++ program that implements three sorting algorithms (Bubble Sort, Selection Sort, and Insertion Sort) to sort the sales data in ascending order.

The program should:

- Print each pass of the algorithm, showing the array after each iteration.
- Display the total number of passes and comparisons made during the sorting process.
- Show the final sorted result of the sales data.

Code:

```
/*Analyzing Sales Data*/

#include<iostream>
#include "printArray.h"
using namespace std;

void bubble(int arr[], int n){

    for(int i = 0 ; i < n-1; i++){
        for(int j = 0; j < n-i-1; j++){
            if(arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
        }
        cout << "After "<<i+1<<" pass: ";
        printIntArray(arr, n);
    }
    cout << "\nFinal sorted array: ";
    printIntArray(arr, n);
}

void selection(int arr[], int n){

    for(int i = 0; i < n-1; i++){
        int smallIndex = i;
```

```
        for(int j = i+1; j < n; j++){
            if(arr[smallIndex] > arr[j]) smallIndex = j;
        }
        swap(arr[i], arr[smallIndex]);

        cout << "After "<<i+1<<" pass: ";
        printIntArray(arr, n);
    }
    cout << "\nFinal sorted array: ";
    printIntArray(arr, n);
}

void insertion(int arr[], int n){

    for(int i = 1; i < 5; i++){ //we are considering first element as sorted
        int curr = arr[i];
        int pre = i - 1;

        while(pre >= 0 && arr[pre] > curr){
            arr[pre+1] = arr[pre];
            pre--;
        }
        arr[pre+1] = curr; // placing current element at it's right place

        cout << "After "<<i+1<<" pass: ";
        printIntArray(arr, n);
    }
    cout << "\nFinal sorted array: ";
    printIntArray(arr, n);
}

int main(){
    int sales[10] = {12500, 97000, 8800, 10200, 13040, 7010, 9800, 10150, 12030,
10500};

    cout << "\nOriginal Array: ";
    printIntArray(sales, 10);

    int choice = 0;
    cout << "\nSelect a sorting algorithm.\n";
    cout << "1. Bubble sort.\n";
    cout << "2. Selection sort.\n";
    cout << "3. Insertion sort.\n";
```

```

cout << "Enter your choice: ";
cin >> choice;

switch(choice){
    case 1:
        cout << "\n\t\t\tBubble Sort:\n";
        bubble(sales, 10);
        break;

    case 2:
        cout << "\n\t\t\tSelection Sort:\n";
        selection(sales, 10);
        break;

    case 3:
        cout << "\n\t\t\tInsertion Sort:\n";
        insertion(sales, 10);
        break;

    default:
        cout << "Invalid choice!\n";
        break;
}
}

```

Output:

```

Original Array: 12500 | 97000 | 8800 | 10200 | 13040 | 7010 | 9800 | 10150 | 12030 | 10500 |

Select a sorting algorithm.
1. Bubble sort.
2. Selection sort.
3. Insertion sort.
Enter your choice: 2

                Selection Sort:
After 1 pass: 7010 | 97000 | 8800 | 10200 | 13040 | 12500 | 9800 | 10150 | 12030 | 10500 |
After 2 pass: 7010 | 8800 | 97000 | 10200 | 13040 | 12500 | 9800 | 10150 | 12030 | 10500 |
After 3 pass: 7010 | 8800 | 9800 | 10200 | 13040 | 12500 | 97000 | 10150 | 12030 | 10500 |
After 4 pass: 7010 | 8800 | 9800 | 10150 | 13040 | 12500 | 97000 | 10200 | 12030 | 10500 |
After 5 pass: 7010 | 8800 | 9800 | 10150 | 10200 | 12500 | 97000 | 13040 | 12030 | 10500 |
After 6 pass: 7010 | 8800 | 9800 | 10150 | 10200 | 10500 | 97000 | 13040 | 12030 | 12500 |
After 7 pass: 7010 | 8800 | 9800 | 10150 | 10200 | 10500 | 12030 | 13040 | 97000 | 12500 |
After 8 pass: 7010 | 8800 | 9800 | 10150 | 10200 | 10500 | 12030 | 12500 | 97000 | 13040 |
After 9 pass: 7010 | 8800 | 9800 | 10150 | 10200 | 10500 | 12030 | 12500 | 13040 | 97000 |

Final sorted array: 7010 | 8800 | 9800 | 10150 | 10200 | 10500 | 12030 | 12500 | 13040 | 97000

```

Task 02:

Consider a simple file management system that display a list of files in a directory, with each file represented by a string containing its name. Design a program that uses the bubble sort algorithm to sort these file names in ascending alphabetical order.

Code:

```
/*File Management */

#include<iostream>
#include<string>
#include "printArray.h"
using namespace std;

void bubble(string s[], int n){
    for(int i = 0; i < n-1; i++){
        for(int j = i; j < n-i-1; j++){
            if(s[j] > s[j+1]) swap(s[j], s[j+1]);
        }
    }
}

int main(){
    string files[5] = {"Report.docx", "Photo.jpg", "Presentation.pptx",
"notes.txt", "Archives.zip"};

    cout << "\nFiles before sorting: ";
    printStringArray(files, 5);

    bubble(files, 5);

    cout << "\nFiles after sorting: ";
    printStringArray(files, 5);
}
```

Output:

```
Files before sorting: Report.docx | Photo.jpg | Presentation.pptx | notes.txt | Archives.zip |
Files after sorting: Photo.jpg | Presentation.pptx | Archives.zip | Report.docx | notes.txt |
PS D:\Hasan\DSA\University\Lab 03 - Sorting Algorithms> █
```

Task 03:

You have 15 clothes to wash and must sort them by color before doing laundry to prevent color bleeding. The clothes should be arranged into groups so that whites come first, followed by darks, and then colored clothes. To organize them efficiently, you will implement the Insertion Sort algorithm to sort the list based on color order before washing.

Code:

```
/*Laundry Clothes Sorting*/

#include<iostream>
using namespace std;

void insertion(char c[], int n){

    for(int i = 1; i < 5; i++){
        int curr = c[i];
        int pre = i - 1;

        while(pre >= 0 && c[pre] < curr){
            c[pre+1] = c[pre];
            pre--;
        }
        c[pre+1] = curr;
    }
}

int main(){
    char colors[5] = {'A', 'H', 'Y', 'C', 'Z'};

    cout << "\nColors before sorting: ";
    for(int i = 0; i < 5; i++) cout << colors[i] << " | ";
    cout << "\n";

    insertion(colors, 5);

    cout << "\nColors after sorting: ";
    for(int i = 0; i < 5; i++) cout << colors[i] << " | ";
    cout << "\n";
}
```

Output:

```
Colors before sorting: A | H | Y | C | Z |
Colors after sorting: Z | Y | H | C | A |
```

Task 04:

Assume that you are developing a Car Rental Service Application that displays a list of available cars. Each car is represented by properties such as the car's name, rental price per day, and type (e.g., sedan, SUV, convertible). Your task is to implement a Selection Sort algorithm with two functions: one that sorts the cars by their type, grouping them into categories such as sedans, SUVs, and convertibles, and another that sorts the cars by price in ascending order, from the least expensive to the most expensive.

Code:

```
/*Organizing Cars by Type and Price*/

#include<iostream>
#include<string>
#include "printArray.h"
using namespace std;

struct Car{
    string name;
    float price;
    string type;
};

void printArray(Car c[], int n){
    for(int i = 0; i < n; i++){
        cout << "Car: " << c[i].name << ", Price: $" << c[i].price << ", Type: "
<< c[i].type << "\n";
    }
}

void selection(Car c[], int n, string byWhatToSort){

    for(int i = 0; i < n-1; i++){
        int smallIndex = i;

        for(int j = i+1; j < n; j++){
            if(byWhatToSort == "price"){
                if(c[j].price < c[smallIndex].price)
                    smallIndex = j;
            }
            else{ // type
                if(c[j].type < c[smallIndex].type)
                    smallIndex = j;
            }
        }
    }
}
```

```
        swap(c[i], c[smallIndex]);
    }
}

int main(){

    Car cars[5] = {
        {"Mehran", 48.25, "Sedan"},
        {"Cultus", 72.00, "SUV"},
        {"Civic", 130.50, "Convertible"},
        {"Corola", 53.40, "Sedan"},
        {"Alto", 110.75, "SUV"}
    };

    int n = 5;

    cout << "Original List of Cars:\n";
    printArray(cars, n);

    selection(cars, n, "type");
    cout << "\nSorted By Type:\n";
    printArray(cars, n);

    selection(cars, n, "price");
    cout << "\nSorted By Price:\n";
    printArray(cars, n);

    return 0;
}
```

Output:

```
Original List of Cars:
Car: Mehran, Price: $48.25, Type: Sedan
Car: Cultus, Price: $72, Type: SUV
Car: Civic, Price: $130.5, Type: Convertible
Car: Corola, Price: $53.4, Type: Sedan
Car: Alto, Price: $110.75, Type: SUV

Sorted By Type:
Car: Civic, Price: $130.5, Type: Convertible
Car: Cultus, Price: $72, Type: SUV
Car: Alto, Price: $110.75, Type: SUV
Car: Corola, Price: $53.4, Type: Sedan
Car: Mehran, Price: $48.25, Type: Sedan

Sorted By Price:
Car: Mehran, Price: $48.25, Type: Sedan
Car: Corola, Price: $53.4, Type: Sedan
Car: Cultus, Price: $72, Type: SUV
Car: Alto, Price: $110.75, Type: SUV
Car: Civic, Price: $130.5, Type: Convertible
```